

SmartCalculator Project

Incremental Build Guide · Weeks 1–8 · Estimated total: ~25 hours

About This Project

SmartCalculator is your single progressive project for the entire 8-week programme. You build it in phases, one phase per week. Each phase adds to what you built in the previous one — so by Week 8 you have a complete, well-tested application.

This guide tells you exactly what to build each week, what the code should do, and what to submit. Keep everything in one Git repository from day one.

What you will build

- **A fully functional CLI calculator** (no frameworks, no web server — plain Java + Scanner)
- Run from the command line, handles arithmetic, edge cases, and custom exceptions
- OOP design: abstract classes, interfaces, and polymorphism
- A static HTML/CSS web front-end to visualise the calculator UI (Week 8)

Ground Rules

Rule	Detail
Language & Phase	Java 21 (pure CLI, no frameworks) for Weeks 3–7. HTML/CSS for Week 8.
Repository	One GitHub repository from Week 1. Push every phase.
Build tool	Maven throughout (set up in Phase 1).
AI tools	Not permitted during lab or build time.
Each phase	Builds on the previous. Never delete earlier code; only extend it.

Phase 1: Project Setup · Week 1 · ~3 hrs

Goal: Create the Maven project, wire up Git, and push a meaningful first commit to GitHub.

What to do

- Open IntelliJ IDEA and choose New Project → Maven. Set the group ID to `com.savoir` and artifact ID to `SmartCalculator`. Select Java 21 SDK.
- Inside `pom.xml`, set the compiler source and target to 21 and the source encoding to UTF-8.
- Open a terminal inside the project folder and run `git init`. Add a `.gitignore` file for Java/Maven (IntelliJ can generate one, or use <https://www.toptal.com/developers/gitignore>).
- Create a `README.md` that contains: the project name, your name, and one line describing what the SmartCalculator will do.
- Make your first commit with the message: `feat: initial Maven project setup`
- Create an empty repository on GitHub and push.

Minimum pom.xml to include

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.savoir</groupId>
  <artifactId>SmartCalculator</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
</project>
```

Deliverable

Submit: Your GitHub repository link. It must show the Maven folder structure, pom.xml, .gitignore, README.md, and at least one commit with a meaningful message.

Phase 2: Requirements & Design · Week 2 · ~3 hrs

Goal: Think before you code. Document what your calculator must do and how you plan to build it.

What to do

- Create a file docs/requirements.md in your repository.
- Apply the **5 Whys technique**: start with 'Why does a user need a calculator app?' and ask 'Why?' five times. Write the chain and the insight it gave you.
- List at least five functional requirements (what the calculator must do) and two non-functional requirements (performance, usability).
- List what you will NOT build (for example: no graphical interface, no server, no database).
- Sketch the planned class structure: which classes you expect to create and their responsibilities. A text diagram is fine.
- Commit with the message: docs: add requirements and design notes

Example requirements format

```
# Functional Requirements
FR1: The calculator shall support +, -, *, /, % operations.
FR2: The user shall enter two numbers and an operator via the CLI.
FR3: The calculator shall run in a loop until the user types 'exit'.
FR4: Division by zero shall display a clear error message, not crash.
FR5: The calculator shall display results to two decimal places.

# Non-Functional Requirements
NF1: Each calculation shall complete and display within 1 second.
NF2: Error messages shall be human-readable.
```

Deliverable

Submit: The updated GitHub repository with docs/requirements.md committed and pushed.

Phase 3: Basic Calculator Logic · Week 3 · ~3 hrs

Goal: Write a working CLI calculator that accepts two numbers and an operator, calculates a result, and loops until the user quits.

What to build

- Create the file `src/main/java/com/savoir/Main.java`.

- Use **Scanner** to read two numbers and an operator from the console.
- Use a **switch expression** (Java 21 arrow syntax) to run the correct calculation.
- Wrap everything in a **while(true)** loop. The loop exits only when the user types **exit** instead of a number.
- Handle division by zero with an if-check and print a clear message — do not crash.
- Print results to two decimal places using `String.format("%.2f", result)`.

Code structure to follow

```
package com.savoir;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=== SmartCalculator ===");
        System.out.println("Type 'exit' to quit.");

        while (true) {
            System.out.print("Enter first number (or 'exit'): ");
            String input = sc.nextLine().trim();
            if (input.equalsIgnoreCase("exit")) break;

            double a = Double.parseDouble(input);
            System.out.print("Enter operator (+ - * / %): ");
            String op = sc.nextLine().trim();
            System.out.print("Enter second number: ");
            double b = Double.parseDouble(sc.nextLine().trim());

            double result = switch (op) {
                case "+" -> a + b;
                case "-" -> a - b;
                case "*" -> a * b;
                case "/" -> {
                    if (b == 0) { System.out.println("Error: division by zero"); yield
Double.NaN; }
                    else yield a / b;
                }
                case "%" -> a % b;
                default -> { System.out.println("Unknown operator"); yield
Double.NaN; }
            };

            if (!Double.isNaN(result))
                System.out.printf("Result: %.2f\n", result);
            System.out.println("Goodbye!");
        }
    }
}
```

Deliverable

Submit: Run the calculator, perform three calculations, and screenshot the console output. Push the code and the screenshot to your repo.

Phase 4: OOP Structure & Code Hygiene · Week 4 · ~3 hrs

Goal: Refactor the single-class code into a proper OOP structure. Apply encapsulation, clean code, and Javadoc.

What to build

- Create a separate **Operation** class that holds the two operands and the operator as private fields.
- Add a **Calculator class** with a public `calculate(Operation op)` method that contains the switch expression. Move the switch logic out of `Main`.
- Add **static utility methods** in `Calculator` for percentage and square root (e.g., `static double squareRoot(double n)`).
- Apply naming conventions: class names are `PascalCase`, method and variable names are `camelCase`. No single-letter variables except loop counters.
- Add Javadoc comments (`/** ... */`) above every public class and method. At minimum include `@param` and `@return` tags.
- Reformat the code using IntelliJ's `Ctrl+Alt+L` (`Cmd+Alt+L` on Mac).

Expected class breakdown

```
com.savoir/
Main.java      ← only reads input and calls Calculator
Calculator.java ← calculate(), squareRoot(), percentage()
Operation.java ← private fields: double a, String operator, double b
                constructor + getters
```

Deliverable

Submit: GitHub link. The repo must have the three-class structure. Run `mvn compile` — no errors. Javadoc must be present on all public methods.

Phase 5: OOP Design — Pillars · Week 5 · ~3 hrs

Goal: Apply inheritance, interfaces, and polymorphism to make the calculator extensible.

What to build

- **Create an abstract class `Operation`** with one abstract method: `abstract double calculate();`
- Create four concrete subclasses that extend `Operation`: **Addition**, **Subtraction**, **Multiplication**, **Division**. Each overrides `calculate()`.
- **Create an interface `Calculable`** with the method `double calculate();` Make each concrete class also implement `Calculable`.
- In `Main` (or a new class), create a `List<Calculable>` containing one of each operation type. Loop through it and print each result — this demonstrates runtime polymorphism.
- Override `toString()` in each concrete class to return something readable, e.g. `"Addition: 5.0 + 3.0 = 8.0"`.

Key code pattern for polymorphism demo

```
List<Calculable> ops = List.of(
    new Addition(10, 4),
    new Subtraction(10, 4),
    new Multiplication(10, 4),
    new Division(10, 4)
);
```

```
for (Calculable op : ops) {
    System.out.println(op.calculate());
}
```

Important: do not remove the working CLI from Phase 3/4. The loop must still work. Integrate the new class hierarchy so the CLI creates the right subclass and calls `calculate()` on it.

Deliverable

Submit: GitHub link. The console must show the polymorphism demo output. Run `mvn compile` — no errors.

Phase 6: Robustness & Code Review · Week 6 · ~3 hrs

Goal: Add custom exceptions, make the calculator safe from bad input, use `BigDecimal` for precision, and self-review your code.

Custom exceptions to create

```
// InvalidOperationException.java
public class InvalidOperationException extends RuntimeException {
    public InvalidOperationException(String message) {
        super(message);
    }
}

// DivisionByZeroException.java
public class DivisionByZeroException extends ArithmeticException {
    public DivisionByZeroException() {
        super("Cannot divide by zero");
    }
}
```

What to do

- Replace the `if (b == 0)` check in `Division` with `throw new DivisionByZeroException();`.
- In `Main`'s input loop, wrap the parsing in a `try-catch` that catches `NumberFormatException` and prints `"Please enter a valid number."` without crashing.
- Add a catch for `InvalidOperationException` and `DivisionByZeroException` so they print a clear message and the loop continues.
- For the `Division` operation, switch from `double` arithmetic to `BigDecimal` to avoid floating-point drift:

```
// Inside Division.calculate()
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal bdA = BigDecimal.valueOf(this.a);
BigDecimal bdB = BigDecimal.valueOf(this.b);
return bdA.divide(bdB, 10, RoundingMode.HALF_UP).doubleValue();
```

- Write a short code-review checklist (in `docs/code-review.md`): at least 5 items you check before committing any code.
- Self-review your own code against this checklist and document one thing you improved.

Deliverable

Submit: GitHub link. The calculator must handle: invalid input, division by zero, and unknown operators — all without crashing. Include `docs/code-review.md`.

Phase 7: Logic, Algorithms & Debugging · Week 7 · ~3 hrs

Goal: Document the calculator's logic visually, add validation using decision logic, and use the debugger to trace a real edge case.

What to do

- **Flowchart:** Draw a flowchart (on paper or a free tool like draw.io) showing the main loop of the calculator: start → read input → is it 'exit'? → read operator → validate → calculate → print → loop. Export as PNG and add to docs/flowchart.png.
- **Pseudocode:** Write pseudocode in docs/pseudocode.md for the square root operation including the edge case: what happens if the input is negative?
- **Input validation:** Add a method isValidOperator(String op) that returns true only for +, -, *, /, %. Use it before creating the Operation object and throw InvalidOperationException if the operator is not in the list.
- **Debugger task:** Set a breakpoint at the line that calls calculate(). Run the program in debug mode. Enter 10 / 0. Use 'Step Into' to trace through Division.calculate() and watch when the exception is thrown. Take a screenshot of the debugger stopped at the exception and save it to docs/debug-screenshot.png.
- Commit with message: feat: add input validation, flowchart and debugger trace

Pseudocode example format

```
FUNCTION squareRoot(number):
  IF number < 0 THEN
    THROW InvalidOperationException('Cannot sqrt a negative number')
  END IF
  RETURN Math.sqrt(number)
END FUNCTION
```

Deliverable

Submit: GitHub link. Must include: docs/flowchart.png, docs/pseudocode.md, docs/debug-screenshot.png, and the working isValidOperator() method.

Phase 8–9: Web UI + Web Awareness · Week 8 · ~3 hrs

Goal: Build a static HTML/CSS calculator interface that looks like a real web calculator. Observe how it differs from your CLI build.

Note: this phase is HTML and CSS only. No JavaScript is required. No backend. No framework. The buttons do not need to work — you are building the visual interface and practising semantic HTML and CSS layout.

File to create

```
src/main/webapp/
  index.html    ← calculator UI
  style.css     ← all styling
```

HTML structure to build

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>SmartCalculator</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="calculator">
    <div class="display">
      <input type="text" id="screen" readonly placeholder="0">
    </div>
    <div class="buttons">
      <!-- Row 1 -->
      <button class="btn operator">C</button>
      <button class="btn operator">+/-</button>
      <button class="btn operator">%</button>
      <button class="btn operator">/</button>
      <!-- Row 2 -->
      <button class="btn">7</button>
      <button class="btn">8</button>
      <button class="btn">9</button>
      <button class="btn operator">*</button>
      <!-- Row 3 -->
      <button class="btn">4</button>
      <button class="btn">5</button>
      <button class="btn">6</button>
      <button class="btn operator">-</button>
      <!-- Row 4 -->
      <button class="btn">1</button>
      <button class="btn">2</button>
      <button class="btn">3</button>
      <button class="btn operator">+</button>
      <!-- Row 5 -->
      <button class="btn zero">0</button>
      <button class="btn">.</button>
      <button class="btn equals">=</button>
    </div>
  </main>
</body>
</html>

```

CSS to apply

- Use Flexbox or Grid to arrange the buttons into a 4-column grid.
- Style .operator buttons with a different background colour (orange or teal).
- Style .equals with an accent colour.
- Make .zero span two columns using grid-column: span 2;
- Add a box shadow and border-radius to the .calculator div so it looks like a physical device.
- Add a media query so the calculator fits on a 375px screen (mobile width).

Reflection task (include in docs/web-reflection.md)

- Open index.html in Chrome. Press F12 to open DevTools. Go to the Network tab, reload, and screenshot the requests.
- Write 3–4 sentences comparing: how the CLI calculator takes input vs how the web UI collects input from a user.
- Write one sentence on where JavaScript would be needed to make the buttons work.

Deliverable

Submit: GitHub link. Must include: `src/main/webapp/index.html`, `style.css`, `docs/web-reflection.md`, and a screenshot of the calculator rendered in the browser saved to `docs/web-screenshot.png`.

Phase Completion Checklist

Use this table to track your progress. Mark each item only when the phase is pushed to GitHub.

Phase	Title	Key Deliverable	GitHub push ✓
1	Project Setup	Maven project + Git repo + README	
2	Requirements & Design	docs/requirements.md	
3	Basic Calculator Logic	Working CLI with loop + switch expression	
4	OOP Structure & Code Hygiene	3-class structure + Javadoc	
5	OOP Pillars	Abstract class + interface + polymorphism demo	
6	Robustness & Code Review	Custom exceptions + BigDecimal + code-review.md	
7	Logic, Algorithms & Debugging	Flowchart + pseudocode + debug screenshot	
8–9	Web UI + Web Awareness	index.html + style.css + web-reflection.md	

Remember: each phase must be reachable by running `git clone` on your repository. Do not submit a zip file — your trainer will check the GitHub commit history.